

Considere la función

$$f(x) = |\sin(x)|$$

en el intervalo $[-1,5;1]$. Determine el lado izquierdo del intervalo resultante de 1 paso del método de la razón áurea para buscar un mínimo de la función en dicho intervalo. Redondee su respuesta a 3 decimales.

In [26]:

```
def fun(x):
    return np.abs(np.sin(x))

a = -1.5
b = 1

r = (-1 + np.sqrt(5))/2

c = a + (1 - r) * (b - a)
d = a + r * (b - a)

# como me piden el limite izquierdo solo me importa el valor que va a tomar a
if fun(c) > fun(d): a = c

print("Limite izquierdo:", np.round(a,3))
```

Limite izquierdo: -0.545

Ejercicio 4:

Considere la ecuación

$$x^3 - x - 1 = 0$$

Usando el método de secante y comenzando de $x=1,8$, estime el número de iteraciones necesarias para alcanzar un error menor a 10^{-3} . En caso de necesitarlo, asuma que la solución a la ecuación es igual a 1,324718.

In [4]:

```
f_1 = lambda x: 3*x**2 - 1 # Derivada primera
f_2 = lambda x: 6 * x      # Derivada segunda

x_f = 1.324718 # Resultado
x_0 = 1.8
R = (1 + np.sqrt(5))/2

err = 1e-3

a = np.abs(f_2(x_f)/(2 * f_1(x_f)))
c = a ** (R - 1)
eo = x_f - x_0

# Resultado de despejar n en la ecuación e_n = (a * |e_o|)^(R^n) / a
niter = np.log(np.log(err * a) / np.log(a * np.abs(eo))) / np.log(R)
print(f'n: {np.ceil(niter)}')
```

n: 5.0

Ejercicio 5:

Considere la matriz:



Sin hacer demasiadas cuentas, explique breve y adecuadamente cuántos valores singulares nulos tiene. No calcule los valores singulares explícitamente.

In [5]:

```
A = np.array([
    [1, -6, -5],
    [2, -6, -4],
    [3,  6,  9]
])

# Llamo c0, c1 y c2 a las columnas, de izquierda a derecha.

# Primero tenemos c0. Después, vemos que c1 no es un múltiplo de c0, con lo cual has
# independientes. Por último, vemos que c2 = c0 + c1, con lo cual la matriz tiene un
# lineal de las otras. Esto hará que, de sus tres autovalores, uno sea nulo.

# Verifico:

print(f'Valores singulares nulos: {np.count_nonzero(np.isclose(np.linalg.svd(A)[1],
```

Valores singulares nulos: 1

Ejercicio 6:

Considere la ecuación diferencial ordinaria (problema de valor inicial)

$$y' = f(t, y), \quad t \in (a, b)$$

$$y(a) = y_0$$

Se resuelve numéricamente usando el método Runge-Kutta 4. Cuando se divide el intervalo en 1.866 sub-intervalos iguales, se obtiene $y(b)$ igual a 0,011. Al duplicar el número de sub-intervalos, se obtiene $y(b)$ igual a 0,014. Estime el número de sub-intervalos necesarios para que el error sea menor a 10^{-6} . La respuesta debe ser un número entero.



In [4]:

```
import numpy as np

n = 4

N1 = 1.866 # cantidad de intervalos -> 1/delta_t
y1 = 0.011

N2 = 2*N1
y2 = 0.014

error = 1e-6

c = (y2 - y1)/((1 - 1/2**n) * (1/N1)**n) #calculo primero la constante

delta_t = (error/c)**(1/n) #el error

n_iter = 1/delta_t #las iteraciones

print(np.ceil(n_iter))
```

15.0

Ejercicio 7:

Considere el sistema lineal de ecuaciones $A \cdot \overrightarrow{x} = \overrightarrow{b}$ donde:



Partiendo de



realice un paso del metodo de Gauss-Siedel. Dé el valor de la última componente de $\overrightarrow{x_1}$ redondeando a 3 decimales.

In [20]:

```
A = np.array([
    [4, 3, 7, 1, 5],
    [2, 4, 3, 2, 5],
    [4, 3, 1, 3, 3],
    [4, 10, 8, 9, 3],
    [3, 9, 10, 6, 10]
])

b = np.array([
    [1],
    [7],
    [1],
    [2],
    [7]
])

h, w = A.shape
x_k = np.zeros(h)

for i in range(h):
    sum_ = np.dot(A[i], x_k) - A[i, i] * x_k[i]
    x_k[i] = (b[i] - sum_) / A[i, i]

print(np.round(x_k[-1],3))
```

2.454

Ejercicio 8:

Considere la función



Redondeando a 3 decimales, dé el punto más cercano al mínimo de entre los resultantes de una sola iteración de Nelder-Mead, comenzando con los puntos:



In [8]:

```
def nelder_mead(f, x_i, maxiter):
    x_ini = x_i.copy()
    y = f(x_ini)

    def get_idx(_x, _y):
        O = _y.argmin()
        B = (_y[np.all(_x != _x[0], axis = 1)]).argmin()
        P = _y.argmax()

    return O, B, P
```

```

O, B, P = get_idx(x_ini, y)

for i in range(maxiter):
    M = (x_ini[O] + x_ini[B])/2

    R = 2 * M - x_ini[P]
    fR = f(np.array([R]))

    if fR < y[0]:
        E = 3 * M - 2 * x_ini[P]
        fE = f(np.array([E]))

        if fE < fR: x_ini[P], y[P] = E, fE
        else: x_ini[P], y[P] = R, fR

    elif fR > y[P]:
        C = np.array([(M + x_ini[P])/2, (M + R)/2])
        fC = f(C)
        mini = fC.argmin()

        if fC[mini] < y[P]: x_ini[P], y[P] = C[mini], fC[mini]

        else:
            x_ini[P], x_ini[B] = (x_ini[O] + x_ini[P])/2, (x_ini[O] + x_ini[B])/2
            y[P], y[B] = f(np.array([x_ini[P], x_ini[B]]))

    else: x_ini[P], y[P] = R, fR

    O, B, P = get_idx(x_ini, y)

return x_ini[0]

f = lambda x: (x[:, 0] - 2)**4 + (x[:, 1] + 2)**2
x_i = np.array([
    [1, 25],
    [2, 10],
    [3, 30]
])

print(f'NM: {np.round(nelder_mead(f, x_i, 1), 3)}')
```

NM: [0 5]

Ejercicio 9:

Considere la iteración de punto fijo

$$y_{k+1} = e^{\{y_k\}^2} - 1.5$$

Determine si la convergencia está garantizada en el intervalo $[-0.41; -0.3]$. Justique brevemente su respuesta.

In [21]:

```

# Verificamos las tres condiciones:

# 1. Que  $g(x) = e^{(x^2)} - 1.5$  sea continua en  $[a, b]$  ---> Lo es.

# 2. Que  $g(x)$  esté dentro de  $[a, b]$  para todo  $x$  perteneciente a  $[a, b]$ .
# Como la función decrece en el intervalo  $[-.41, -.3]$ , entonces sus límites serán
# máximo y mínimo, respectivamente. Y éstos dan  $-.317$  y  $-0.4058$ , que pertenecen
```

```
# 3. Que exista  $K < 1$  tal que  $|g'(x)| \leq K$  para todo  $x$  en  $[a, b]$ .
# Como  $g(x) = e^{(x^2)} - 1.5$ , entonces  $g'(x) = 2x * e^{(x^2)}$ .
# Entonces  $|g'(x)|$  en  $[-.41, -.3]$  será siempre menor o igual a  $2 * .41 * e^{(.41^2)}$ 

# Por lo que la convergencia está garantizada.
```

Ejercicio 10:

Considere la función

$$f(x) = \cos(x)$$

en el intervalo $[2.5; 3.5]$. Utilizando interpolación cuadrática con nodos equi-espaciados en dicho intervalo, encuentre una aproximación al mínimo de la función. Redondee su respuesta a 3 decimales.

In [10]:

```
f = lambda x: np.cos(x)

a = 2.5
c = 3.5
b = (a + c)/2

h = (b - a)

ya, yb, yc = f(a), f(b), f(c)
hmin = (4 * yb - 3 * ya - yc) / (4 * yb - 2 * ya - 2 * yc) * h

print(f'Mínimo: {np.round(a + hmin, 3)}')
```

Mínimo: 3.14